

ADVANCED WEB PROGRAMMING
EXPORT & IMPORT



KHEN MUHAMMAD CAHYO

244107023002

TI-21

STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT

TASK 1 – IMPORT DATA

- User Level**

Import Data (Excel)

File

Choose File

Import-Level.xlsx

Upload

Success

Level data imported successfully.

OK

4	EXM1	Example 1	Details Update Delete	Details Update Delete
5	EXM2	Example 2	Details Update Delete	Details Update Delete

1	EXM1	Example 1	
2	EXM2	Example 2	

- User**

Import Data (Excel)

File

Choose File

Import-User.xlsx

Upload

Success

User data imported successfully.

OK

4	contoh1	Contoh 1	Manager	Details Update Delete	Details Update Delete
5	contoh2	Contoh 2	Manager	Details Update Delete	Details Update Delete

	A	B	C	D	
	2	contoh1	Contoh 1	contoh	
	2	contoh2	Contoh 2	contoh	

- Category

Import Data (Excel)

File

Choose File

Import-Category.xlsx

Upload

Success

Category data imported successfully.

OK

6	CTH1	Contoh Category 1	DetailsUpdateDelete	DetailsUpdateDelete
7	CTH2	Contoh Category 2	DetailsUpdateDelete	DetailsUpdateDelete

A	B	C
CTH1	Contoh Category 1	
CTH2	Contoh Category 2	

- Item

Import Data (Excel)

File

Choose File

Import-Item.xlsx

Upload

Success

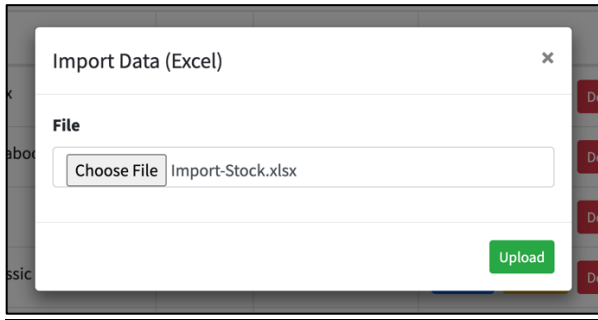
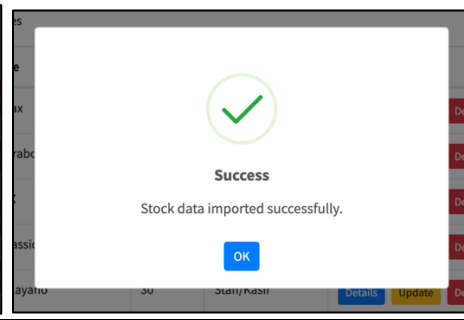
Item data imported successfully.

OK

ID	Code	Name	Category	Actions	Actions AJAX
11	SNK09	Nike Dunk Low	Sneaker Pria	DetailsUpdateDelete	DetailsUpdateDelete
12	SNK010	Nike Air Jordan	Sneaker Pria	DetailsUpdateDelete	DetailsUpdateDelete

A	B	C	D	E
1	SNK09	Nike Dunk Lo	1200000	1300000
1	SNK010	Nike Air Jord	1500000	2000000

- **Stock**

ID	Item Name	QTY	Created by	Actions	Actions AJAX
11	Nike Air Max	5	Staff/Kasir	Details Update Delete	Details Update Delete
12	Adidas Ultraboost	5	Staff/Kasir	Details Update Delete	Details Update Delete

	A	B	C	D
1		3	5	
2		3	5	

- **Explanation**

Because all of this feature has same code, I will explain it from one entity (User Level). For this import data from excel feature, I used Laravel Excel library from maatwebsite, and here is explanation from each steps I did.

- **Creating Import class**

```

Windsurf: Refactor | Explain
class LevelImport implements ToModel
{
    /**
     * @param array $row
     *
     * @return \Illuminate\Database\Eloquent\Model|null
     */
    Windsurf: Refactor | Explain | X
    public function model(array $row)
    {
        return new Level([
            'level_code' => $row[0],
            'level_name' => $row[1],
        ]);
    }
}

```

A required class used for mapping data into the database in the form of a model. This is a feature of Laravel Excel and is created with the command `php artisan make:import LevelImport --model=Level`.

- **Creating Controller**

```
Windsurf: Refactor | Explain | Generate Function Comment | X
public function importExcelAjax(ImportExcelRequest $request) {
    if (!$request->validated()) {
        return response()->json([
            'success' => false,
            'message' => 'Failed to import Excel file. Please check again your data.',
            'errors' => $request->errors(),
        ]);
    }

    $excelFile = $request->file('file');

    try {
        Excel::import(new LevelImport, $excelFile);

        return response()->json([
            'success' => true,
            'message' => 'Level data imported successfully.',
        ]);
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => 'Failed to import Excel file.',
            'errors' => 'Error: ' . $e->getMessage(),
        ]);
    }
}
```

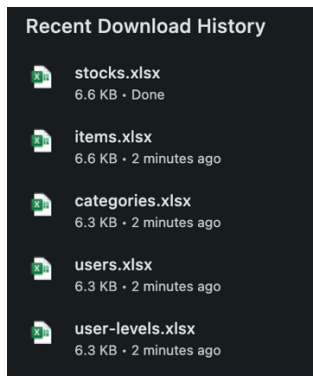
A method that used to import data from an Excel file asynchronously via AJAX. This method first validates the request using ImportExcelRequest. If the validation fails, it returns a JSON response with the failed status and error details. If validation is successful, the Excel file is retrieved from the input file, then processed using Laravel Excel from maatwebsite by running the LevelImport class to handle the import process.

■ AJAX Request

```
<div class="modal fade" id="importExcelLevelAjaxModal" tabindex="-1" role="dialog"
aria-labelledby="importExcelLevelAjaxModallabel" aria-hidden="true">
<div class="modal-dialog modal-dialog-centered" role="document">
<div class="modal-content">
<div class="modal-header">
<h5 class="modal-title" id="importExcelLevelAjaxModallabel">Import Data (Excel)</h5>
<button type="button" class="close" data-dismiss="modal" aria-label="Close">
<span aria-hidden="true">&times;</span>
</button>
</div>
<div>
<form action="{{ route('levels.import-excel-ajax') }}" method="POST" id="importExcelLevelAjaxForm">
<csrf
<div class="modal-body">
<div class="form-group">
<label for="file" class="form-label">File</label>
<input class="form-control" type="file" id="file" name="file">
<small class="error-text text-danger" id="error-file"></small>
</div>
</div>
<div class="modal-footer">
<button type="submit" class="btn btn-success btn-sm">Upload</button>
</div>
</form>
</div>
</div>
</div>
```

```
@push('scripts')
<script>
$(document).ready(function() {
$('#importExcelLevelAjaxForm').validate({
rules: {
file: {
required: true,
extension: "xlsx|xls|csv"
},
},
submitHandler: function(form) {
$.ajax({
url: form.action,
type: form.method,
data: new FormData(form),
contentType: false,
processData: false,
success: function(response) {
if (response.success) {
$('#importExcelLevelAjaxModal').modal('hide');
Swal.fire({
icon: 'success',
title: 'Success',
text: response.message
});
$('#levelsTable').DataTable().ajax.reload();
} else {
$('#error-text').text('');
$.each(response.errors, function(key, value) {
$('#error-' + key).text(value);
});
Swal.fire({
icon: 'error',
title: 'Error',
text: response.message
});
}
},
error: function(xhr) {
console.log(xhr.responseText);
}
});
return false;
},
errorElement: 'small',
errorPlacement: function(error, element) {
error.addClass('invalid-feedback');
element.closest('.form-group').append(error);
},
highlight: function(element, errorClass, validClass) {
$(element).addClass('is-invalid');
},
unhighlight: function(element, errorClass, validClass) {
$(element).removeClass('is-invalid');
}
});
});
</script>
@endpush
```

TASK 2 – EXPORT DATA (EXCEL)



- **User Level**

	A	B	C	D	E	
	ID	Code	Name			
	1	ADM	Administrator			
	2	MNG	Manager			
	3	STF	Staff/Kasir			
	8	EXM1	Example 1			
	9	EXM2	Example 2			

- **User**

	A	B	C	D	E	F
	ID	Username	Name	Level		
	1	admin	Administra	Administrator		
	2	manager	Manager	Manager		
	3	staff	Staff/Kasir	Staff/Kasir		
	8	contoh1	Contoh 1	Manager		
	9	contoh2	Contoh 2	Manager		

- **Category**

	ID	Code	Name			
	1	SNK1	Sneaker Pria			
	2	SNK2	Sneaker Wanita			
	3	OR1	Sepatu Olahraga			
	4	CAS1	Casual			
	5	SND1	Sandal Pria			
	6	CTH1	Contoh Category 1			
	7	CTH2	Contoh Category 2			

- Item

ID	Category	Code	Name	Buy Price	Sell Price
1	Sneaker Pria	SNK101	Nike Air Ma	500000	750000
2	Sneaker Pria	SNK102	Adidas Ultr	600000	850000
3	Sneaker Wa	SNK201	Puma RS-X	550000	800000
4	Sneaker Wa	SNK202	Reebok Cla	400000	650000
5	Sepatu Olal	OR101	Asics Gel Ka	700000	1000000
6	Sepatu Olal	OR102	Mizuno Wa	650000	950000
7	Casual	CAS101	Vans Old Sk	450000	700000
8	Casual	CAS102	Converse C	400000	600000
9	Sandal Pria	SND101	Eiger Sanda	300000	500000
10	Sandal Pria	SND102	Skechers Sa	350000	550000
11	Sneaker Pria	SNK09	Nike Dunk I	1200000	1300000
12	Sneaker Pria	SNK010	Nike Air Jor	1500000	2000000

- Stock

ID	Item Code	Item Name	Quantity
1	SNK101	Nike Air Ma	10
2	SNK102	Adidas Ultr	5
3	SNK201	Puma RS-X	20
4	SNK202	Reebok Cla	15
5	OR101	Asics Gel Ka	30
6	OR102	Mizuno Wa	20
7	CAS101	Vans Old Sk	10
8	CAS102	Converse C	10
9	SND101	Eiger Sanda	10
10	SND102	Skechers Sa	5
11	SNK101	Nike Air Ma	5
12	SNK102	Adidas Ultr	5

- **Explanation**

Because all of this feature has same code, I will explain it from one entity (User Level). For this export data from excel feature, I used Laravel Excel library from maatwebsite, and here is explanation from each steps I did.

- **Creating Export Class**

```
class LevelExport implements FromCollection, WithHeadings, WithStyles
{
    /**
     * @return \Illuminate\Support\Collection
     */
    Windsurf: Refactor | Explain | X
    public function collection()
    {
        return Level::select('level_id', 'level_code', 'level_name')->get();
    }

    Windsurf: Refactor | Explain | Generate Function Comment | X
    public function headings(): array
    {
        return [
            'ID',
            'Code',
            'Name',
        ];
    }

    Windsurf: Refactor | Explain | Generate Function Comment | X
    public function styles(Worksheet $sheet)
    {
        return [
            1 => ['font' => ['bold' => true]],
        ];
    }
}
```

This class implements three interfaces: FromCollection to export data from the Eloquent collection (Level::select(...)->get()), WithHeadings to specify column headings on the first row of Excel (i.e. 'ID', 'Code', and 'Name'), and WithStyles to add styles to the worksheet, where in this example, the first row is styled with bold text to highlight the headings.

- **Creating Controller**

```
Windsurf: Refactor | Explain | Generate Function Comment | X
public function exportExcel() {
    return Excel::download(new LevelExport, 'user-levels.xlsx');
}
```

The exportExcel() function is responsible for exporting data into an Excel file and downloading it directly when called. This function utilizes the Excel facade of Laravel Excel to process the export, using an instance of the LevelExport class as the data source and export configuration. The exported file will be named user-levels.xlsx.

- **Routing & Download Action**

```
Route::get('/', [UserController::class, 'page'])->name('users.page');
Route::get('/export-excel', [UserController::class, 'exportExcel'])->name('users.export-excel');

data-can-get= #import <ExcelStockAjaxModule> <import <data (id=1)> button>
<a href="{{ route('stocks.export-excel') }}" class="btn btn-primary btn-sm ml-0 ml-md-2">Export Data</a>
```

TASK 3 – EXPORT DATA (PDF)

- User Level

Export PDF - Levels Data

1 / 1100%+

1

MINISTRY OF EDUCATION, CULTURE, RESEARCH AND TECHNOLOGY

POLYTECHNIC NEGERI MALANG

Jl. Soekarno-Hatta No. 9 Malang 65141

Phone (0341) 404424 Pes. 101-105, 0341-404420, Fax. (0341) 404420

Page: www.polinema.ac.id

LEVELS DATA REPORT

No	Code	Name
1	ADM	Administrator
2	MNG	Manager
3	STF	Staff/Kasir
4	EXM1	Example 1
5	EXM2	Example 2

- User

Export PDF - Users Data

1 / 1100%+

1

MINISTRY OF EDUCATION, CULTURE, RESEARCH AND TECHNOLOGY

POLYTECHNIC NEGERI MALANG

Jl. Soekarno-Hatta No. 9 Malang 65141

Phone (0341) 404424 Pes. 101-105, 0341-404420, Fax. (0341) 404420

Page: www.polinema.ac.id

USERS DATA REPORT

No	Name	Username	Level
1	Administrator	admin	Administrator
2	Manager	manager	Manager
3	Staff/Kasir	staff	Staff/Kasir
4	Contoh 1	contoh1	Manager
5	Contoh 2	contoh2	Manager

- Category

Export PDF - Categories Data

1 / 1100%+

1

MINISTRY OF EDUCATION, CULTURE, RESEARCH AND TECHNOLOGY

POLYTECHNIC NEGERI MALANG

Jl. Soekarno-Hatta No. 9 Malang 65141

Phone (0341) 404424 Pes. 101-105, 0341-404420, Fax. (0341) 404420

Page: www.polinema.ac.id

CATEGORIES DATA REPORT

No	Code	Name
1	SNK1	Sneaker Pria
2	SNK2	Sneaker Wanita
3	OR1	Sepatu Olahraga
4	CAS1	Casual
5	SND1	Sandal Pria
6	CTH1	Contoh Category 1
7	CTH2	Contoh Category 2

- Item

Export PDF - Items Data

1 / 1 | - 100% + |

1

MINISTRY OF EDUCATION, CULTURE, RESEARCH AND TECHNOLOGY
POLYTECHNIC NEGERI MALANG
Jl. Soekarno-Hatta No. 9 Malang 65141
Phone (0341) 404424 Pes. 101-105, 0341-404420, Fax, (0341) 404420
Page: www.polinema.ac.id

ITEMS DATA REPORT

No	Code	Name	Category	Buy Price	Sell Price
1	SNK101	Nike Air Max	Sneaker Pria	500000	750000
2	SNK102	Adidas Ultraboost	Sneaker Pria	600000	850000
3	SNK201	Puma RS-X	Sneaker Wanita	550000	800000
4	SNK202	Reebok Classic	Sneaker Wanita	400000	650000
5	OR101	Asics Gel Kayano	Sepatu Olahraga	700000	1000000
6	OR102	Mizuno Wave Rider	Sepatu Olahraga	650000	950000
7	CAS101	Vans Old Skool	Casual	450000	700000
8	CAS102	Converse Chuck Taylor	Casual	400000	600000
9	SND101	Eiger Sandal Outdoor	Sandal Pria	300000	500000
10	SND102	Skechers Sandal	Sandal Pria	350000	550000
11	SNK09	Nike Dunk Low	Sneaker Pria	1200000	1300000
12	SNK010	Nike Air Jordan	Sneaker Pria	1500000	2000000

- Stock

Export PDF - Items Data

1 / 1 | - 100% + |

1

MINISTRY OF EDUCATION, CULTURE, RESEARCH AND TECHNOLOGY
POLYTECHNIC NEGERI MALANG
Jl. Soekarno-Hatta No. 9 Malang 65141
Phone (0341) 404424 Pes. 101-105, 0341-404420, Fax, (0341) 404420
Page: www.polinema.ac.id

ITEMS DATA REPORT

No	Item Name	Quantity
1	Nike Air Max	10
2	Adidas Ultraboost	5
3	Puma RS-X	20
4	Reebok Classic	15
5	Asics Gel Kayano	30
6	Mizuno Wave Rider	20
7	Vans Old Skool	10
8	Converse Chuck Taylor	10
9	Eiger Sandal Outdoor	10
10	Skechers Sandal	5
11	Nike Air Max	5
12	Adidas Ultraboost	5

- **Explanation**

```
Windsurf: Refactor | Explain | Generate Function Comment | X
public function exportPdf() {
    $levels = Level::all();

    $pdf = Pdf::loadView('pages.level.export-pdf', compact('levels'));
    $pdf->setPaper('a4', 'potrait');
    $pdf->setOption('isRemoteEnabled', true);
    $pdf->render();

    return $pdf->stream('Levels data ' . Carbon::now()->format('d-m-Y') . '.pdf');
}
```

Because all of this feature has same code, I will explain it from one entity (User Level). exportPdf itself is a function used to export level data into PDF format. This function first retrieves all the data from the Level model using Level::all(), then uses the PDF library (likely barryvdh/laravel-dompdf) to load view pages.level.export-pdf with the retrieved data. After that, the PDF is set to use A4 paper size and portrait orientation, as well as enabling the isRemoteEnabled option so that it can retrieve external resources if needed. Next, the render() function is executed to process the contents of the PDF, and finally the PDF file is displayed directly in the browser using the stream() method with a filename that includes the current date.

TASK 4 – UPLOAD PROFILE PICTURE

- Results

Profile

[Settings](#) / [Profile](#)

Profile Picture



Choose File

No file chosen

Update Profile Picture

Account Informations

Username

admin

Name

Administrator

Level

Administrator

Password

Name

Update Account Informations

• Codes

```
class ProfilePictureRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    Windows::Reflector ( Explain ) <
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     */
    * @return array|string, \Illuminate\Contracts\Validation\ValidationRule|array|string
    Windows::Reflector ( Explain ) <
    public function rules(): array
    {
        return [
            'profile_picture' => 'required|names:jpeg,png,jpg|max:2048',
        ];
    }
}
```

```
Windows::Reflector ( Explain ) Generate Function Comment ( <
public function updateProfilePicture(ProfilePictureRequest $request)
{
    $user = User::find(auth()->id());

    if ($user->profile_picture && Storage::disk('public')->exists($user->profile_picture)) {
        Storage::disk('public')->delete($user->profile_picture);
    }

    $filePath = $request->file('profile_picture')->store('user-profile', 'public');
    $user->update([
        'profile_picture' => $filePath
    ]);

    return redirect()->back()->with('success', 'Profile picture updated successfully.');
```

```
Route::prefix('settings')->group(function () {
    Route::prefix('profile')->group(function () {
        Route::get('', [ProfileController::class, 'index'])->name('settings.profile');
        Route::patch('/profile-picture/update', [ProfileController::class, 'updateProfilePicture'])->name('settings.profile.update-profile-picture');
    });
});
```

```

{{-- Profile Picture --}}
<div class="col-12 col-md-6">
    <div class="card">
        <div class="card-header">
            <p class="fs-5 my-0">Profile Pictures</p>
        </div>
        <form action="{{ route('settings.profile.update-profile-picture') }}" method="POST"
            enctype="multipart/form-data">
            @csrf
            @method('PATCH')
            <div class="card-body">
                @if (session('success'))
                    <div class="alert alert-success alert-dismissible fade show" role="alert">
                        <strong>Success!</strong> {{ session('success') }}
                        <button type="button" class="close" data-dismiss="alert" aria-label="Close">
                            <span aria-hidden="true">&times;</span>
                        </button>
                    </div>
                @elseif (session('error'))
                    <div class="alert alert-danger alert-dismissible fade show" role="alert">
                        <strong>Success!</strong> {{ session('success') }}
                        <button type="button" class="close" data-dismiss="alert" aria-label="Close">
                            <span aria-hidden="true">&times;</span>
                        </button>
                    </div>
                @endif

                @if ($user->profile_picture)
                    <div class="d-flex align-items-center justify-content-center rounded-circle mx-auto border"
                        style="width: 124px; height: 124px; overflow: hidden;">
                        
                    </div>
                @else
                    <div class="d-flex align-items-center justify-content-center rounded-circle mx-auto border"
                        style="width: 124px; height: 124px; background-color: #whitesmoke;">
                        <p class="my-0" style="font-size: 40px;">{{ auth()->user()->name[0] }}</p>
                    </div>
                @endif

                <div class="form-group mt-4">
                    <input type="file" class="form-control" name="profile_picture">
                    @error('profile_picture')
                        <small class="text-danger">{{ $message }}</small>
                    @enderror
                </div>
            </div>
            <div class="card-footer">
                <button type="submit" class="btn btn-primary btn-sm">Update Profile Picture</button>
            </div>
        </form>
    </div>
</div>
```

- **Explanation**

In the ProfilePictureRequest section, there is validation to ensure that the uploaded file is of jpeg, png, or jpg type and its size does not exceed 2MB. Inside the ProfileController, the updateProfilePicture function receives the validated request, then checks if the current user already has a previous profile picture and removes it from storage if it exists. After that, the new file is saved to the user-profile folder in the public disk, and its path is stored in the user's profile_picture field. The view displays file-type input with validation error handling, and the route uses the PATCH method to process the profile photo update request via the URL /profile-picture/update.

